CTMSCS/DFIS SIII P4/P5 EL3 Database Security/ Forensics **Odd 2026**

Lecture Note#2: Joining, Grouping and Subqueries

Lecturer: Dr. Himanshu B Patel *Scribes: M.Sc CS/DFIS 2nd year, Semester III*

Note 1: *LaTeX template courtesy of UC Berkeley EECS dept.*
Note 2: *This documet might contain hyperlinks that require active internet connection.*
Disclaimer: *This lecture note is prepared for the students of the School of Cybersecuirity and Digital Forensics, National Forensic Sciences University and has not been subjected to the usual scrutiny reserved for formal publications. They may be distributed outside this class only with the permission of the Instructor.*

2.1 Motivation and Learning Outcomes

Real-world queries rarely involve a single table or a flat list of rows. We often need to:

- **Summarise** data (How many employees per department? Average salary?),
- **Combine** data from multiple tables (Which employees are working on which projects?),
- **Ask questions about questions** (Employees whose salary is above the average salary of their department).

This unit focuses on three powerful tools of SQL:

1. **Group functions** and the `GROUP BY`/`HAVING` clauses,
2. **Joins** to display data from multiple tables,
3. **Subqueries** (including correlated subqueries and the `WITH` clause).

Pre-requisite: Common Practice Schema

We will continue to use the same schema as in **Unit III**:

- `DEPARTMENT(DEPT_ID, DEPT_NAME, LOCATION)`
- `EMPLOYEE(EMP_ID, FIRST_NAME, LAST_NAME, EMAIL, PHONE_NO, HIRE_DATE, JOB_TITLE, SALARY, COMMISSION DEPT_ID, MANAGER_ID, IS_ACTIVE)`
- `PROJECT(PROJ_ID, PROJ_NAME, START_DATE, END_DATE, BUDGET, DEPT_ID)`
- `EMP_PROJECT(EMP_ID, PROJ_ID, ROLE, ALLOCATED_HOURS)`

Instruction to students: If you have not yet run the schema script from Unit III, open https://livesql.oracle.com, log in, and execute the creation/population script from the previous lecture note. All examples in this note assume that data is already present.

[0]These notes are partially based on the book "Introduction to Languages and the Theory of Computation" by John C Martin (Fourth Edition)

2.2 Aggregate (Group) Functions

2.2.1 What Are Group Functions?

Group functions (or **aggregate functions**) operate on a set of rows and return a **single value**:

- `COUNT` – number of rows,
- `SUM` – total of a numeric column,
- `AVG` – average of a numeric column,
- `MIN` – minimum value,
- `MAX` – maximum value.

They are often used with:

- no `GROUP BY` (entire table as one group), or
- with `GROUP BY` (one group per distinct value of some column).

2.2.2 Aggregate Functions Without GROUP BY

Example 2.1 (Basic Salary Statistics)

```
-- Overall statistics of EMPLOYEE salaries
SELECT
COUNT(*) AS TOTAL_EMPLOYEES,
MIN(SALARY) AS MIN_SALARY,
MAX(SALARY) AS MAX_SALARY,
AVG(SALARY) AS AVG_SALARY
FROM EMPLOYEE;
```

Example 2.2 (Total Allocated Hours on All Projects)

```
SELECT
SUM(ALLOCATED_HOURS) AS TOTAL_HOURS
FROM EMP_PROJECT;
```

2.2.3 Aggregate Functions With GROUP BY

GROUP BY divides rows into groups, then aggregate functions are applied to each group.

Example 2.3 (Average Salary by Department)

```
SELECT
DEPT_ID,
COUNT(*) AS EMP_COUNT,
AVG(SALARY) AS AVG_SALARY
FROM EMPLOYEE
GROUP BY DEPT_ID;
```

Better: join with `DEPARTMENT` to show department names (we will formally discuss joins later, but you can already read this):

```
SELECT
d.DEPT_NAME,
COUNT(e.EMP_ID) AS EMP_COUNT,
ROUND(AVG(e.SALARY), 2) AS AVG_SALARY
FROM DEPARTMENT d
LEFT JOIN EMPLOYEE e
ON d.DEPT_ID = e.DEPT_ID
GROUP BY d.DEPT_NAME;
```

2.2.4 Rules for GROUP BY

- Every column in the `SELECT` list that is *not* inside an aggregate function must appear in the `GROUP BY` clause.
- You can group by one or more columns.
- The result has one row per distinct combination of the grouping columns.

Example 2.4 (Projects Count Per Department and Location)

```
SELECT
d.LOCATION,
d.DEPT_NAME,
COUNT(p.PROJ_ID) AS PROJECT_COUNT
FROM DEPARTMENT d
LEFT JOIN PROJECT p
ON d.DEPT_ID = p.DEPT_ID
GROUP BY d.LOCATION, d.DEPT_NAME
ORDER BY d.LOCATION, d.DEPT_NAME;
```

2.2.5 HAVING Clause

WHERE filters individual rows *before* grouping. **HAVING** filters *groups after* aggregate functions are applied.

Example 2.5 (Departments with Average Salary Above 80,000)

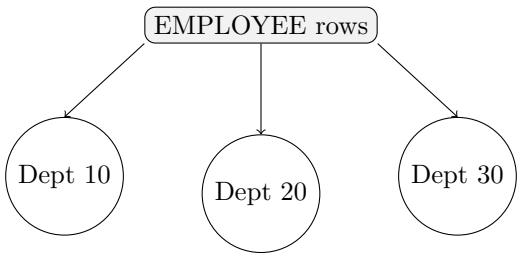
```
SELECT
d.DEPT_NAME,
AVG(e.SALARY) AS AVG_SALARY
FROM DEPARTMENT d
JOIN EMPLOYEE e
ON d.DEPT_ID = e.DEPT_ID
GROUP BY d.DEPT_NAME
HAVING AVG(e.SALARY) > 80000
ORDER BY AVG_SALARY DESC;
```

Example 2.6 (Projects with At Least 150 Allocated Hours)

```
SELECT
p.PROJ_NAME,
SUM(ep.ALLOCATED_HOURS) AS TOTAL_HOURS
```

```
FROM PROJECT p
JOIN EMP_PROJECT ep
ON p.PROJ_ID = ep.PROJ_ID
GROUP BY p.PROJ_NAME
HAVING SUM(ep.ALLOCATED_HOURS) >= 150;
```

2.2.6 Grouping Concept



Each group → one result row after applying AVG/SUM/COUNT

Figure 2.1: Conceptual view of GROUP BY: rows partitioned into groups by department

2.3 Joins: Displaying Data from Multiple Tables

2.3.1 Why Do We Need Joins?

Relational design *normalises* data into multiple tables. To answer real queries, we often need to combine them.

Typical examples:

- Employees along with their department names.
- Projects along with their owning department and assigned employees.
- Employees and their managers (self-join).

2.3.2 Types of Joins (Logically)

- **INNER JOIN**: rows that match in both tables.
- **LEFT OUTER JOIN**: all rows from left table; matching rows from right.
- **RIGHT OUTER JOIN**: all rows from right table; matching left.
- **FULL OUTER JOIN**: all rows from both tables; matched where possible.

- **CROSS JOIN**: Cartesian product.
- **SELF JOIN**: join a table to itself.

2.3.3 ANSI Join Syntax

We will use the modern ANSI syntax:

```
SELECT ...
FROM table1 t1
JOIN table2 t2
ON t1.column = t2.column;
```

2.3.4 Inner Join Examples

Example 2.7 (Employees with Their Department Names)

```
SELECT
e.EMP_ID,
e.FIRST_NAME,
e.LAST_NAME,
d.DEPT_NAME,
d.LOCATION
FROM EMPLOYEE e
JOIN DEPARTMENT d
ON e.DEPT_ID = d.DEPT_ID;
```

Example 2.8 (Employee-Project Assignments)

```
SELECT
e.FIRST_NAME || ' ' || e.LAST_NAME AS EMP_NAME,
p.PROJ_NAME,
ep.ROLE,
ep.ALLOCATED_HOURS
FROM EMPLOYEE e
JOIN EMP_PROJECT ep
ON e.EMP_ID = ep.EMP_ID
JOIN PROJECT p
ON ep.PROJ_ID = p.PROJ_ID;
```

2.3.5 Self Join: Manager and Subordinate

```
SELECT
e.FIRST_NAME || ' ' || e.LAST_NAME AS EMP_NAME,
m.FIRST_NAME || ' ' || m.LAST_NAME AS MANAGER_NAME
FROM EMPLOYEE e
LEFT JOIN EMPLOYEE m
ON e.MANAGER_ID = m.EMP_ID;
```

2.3.6 Outer Join Examples

Example 2.9 (LEFT OUTER JOIN: All Departments, Employees If Any)

```
SELECT
d.DEPT_NAME,
e.FIRST_NAME,
e.LAST_NAME
FROM DEPARTMENT d
LEFT JOIN EMPLOYEE e
ON d.DEPT_ID = e.DEPT_ID
ORDER BY d.DEPT_NAME;
```

If a department has **no employees**, the employee columns will be `NULL`, but the department still appears.

Example 2.10 (RIGHT OUTER JOIN: All Projects, Employees If Any)

```
SELECT
p.PROJ_NAME,
e.FIRST_NAME,
e.LAST_NAME,
ep.ROLE
FROM EMPLOYEE e
RIGHT JOIN EMP_PROJECT ep
ON e.EMP_ID = ep.EMP_ID
RIGHT JOIN PROJECT p
ON ep.PROJ_ID = p.PROJ_ID;
```

Example 2.11 (FULL OUTER JOIN: Departments and Projects)

```
SELECT
d.DEPT_NAME,
p.PROJ_NAME
FROM DEPARTMENT d
FULL OUTER JOIN PROJECT p
ON d.DEPT_ID = p.DEPT_ID
ORDER BY d.DEPT_NAME, p.PROJ_NAME;
```

2.4 Subqueries

2.4.1 What is a Subquery?

A **subquery** is a query nested inside another SQL statement. It can appear in:

- `WHERE` clause,
- `FROM` clause,
- `SELECT` list.

Classifications:

- **Single-row subquery** – returns exactly one row.

- **Multiple-row subquery** – returns many rows.
- **Multiple-column subquery** – returns multiple columns.
- **Correlated subquery** – subquery depends on each row from outer query.

2.4.2 Single-Row Subquery

Example 2.12 (Employees with Salary Above the Overall Average)

```
SELECT
FIRST_NAME,
LAST_NAME,
SALARY
FROM EMPLOYEE
WHERE SALARY > (
SELECT AVG(SALARY) FROM EMPLOYEE
);
```

The inner query returns a **single value** (average salary), and the outer query uses it in a comparison.

2.4.3 Multiple-Row Subqueries

Example 2.13 (Employees in Departments Located in Gandhinagar)

```
SELECT
FIRST_NAME,
LAST_NAME,
DEPT_ID
FROM EMPLOYEE
WHERE DEPT_ID IN (
SELECT DEPT_ID
FROM DEPARTMENT
WHERE LOCATION = 'Gandhinagar'
);
```

Example 2.14 (Departments With Any Employee Salary Greater Than 1,00,000) *Using `ANY` with a scalar comparison:*

```
SELECT DEPT_ID, DEPT_NAME
FROM DEPARTMENT
WHERE DEPT_ID = ANY (
SELECT DEPT_ID
FROM EMPLOYEE
WHERE SALARY > 100000
);
```

2.4.4 Correlated Subqueries

A **correlated subquery** is executed once for each row of the outer query.

Example 2.15 (Employees Earning More Than the Average of Their Own Department)

```
SELECT
e.FIRST_NAME,
e.LAST_NAME,
e.SALARY,
e.DEPT_ID
FROM EMPLOYEE e
WHERE e.SALARY > (
SELECT AVG(e2.SALARY)
FROM EMPLOYEE e2
WHERE e2.DEPT_ID = e.DEPT_ID
);
```

Example 2.16 (Departments With Average Salary Greater Than Overall Average)

```
SELECT
d.DEPT_NAME,
AVG(e.SALARY) AS DEPT_AVG_SAL
FROM DEPARTMENT d
JOIN EMPLOYEE e
ON d.DEPT_ID = e.DEPT_ID
GROUP BY d.DEPT_NAME
HAVING AVG(e.SALARY) > (
SELECT AVG(SALARY)
FROM EMPLOYEE
);
```

2.4.5 EXISTS and NOT EXISTS

`EXISTS` checks whether the subquery returns at least one row.

Example 2.17 (Departments With At Least One Employee)

```
SELECT d.DEPT_ID, d.DEPT_NAME
FROM DEPARTMENT d
WHERE EXISTS (
SELECT 1
FROM EMPLOYEE e
WHERE e.DEPT_ID = d.DEPT_ID
);
```

Example 2.18 (Projects With No Employees Assigned)

```
SELECT p.PROJ_ID, p.PROJ_NAME
FROM PROJECT p
WHERE NOT EXISTS (
SELECT 1
FROM EMP_PROJECT ep
WHERE ep.PROJ_ID = p.PROJ_ID
);
```

2.4.6 Subqueries in the FROM Clause (Inline Views)

Subqueries can act like temporary tables (also called **inline views**).

Example 2.19 (Department-wise Aggregates Using an Inline View)

```
SELECT
d.DEPT_NAME,
v.EMP_COUNT,
v.AVG_SALARY
FROM DEPARTMENT d
JOIN (
SELECT
DEPT_ID,
COUNT(*) AS EMP_COUNT,
AVG(SALARY) AS AVG_SALARY
FROM EMPLOYEE
GROUP BY DEPT_ID
) v
ON d.DEPT_ID = v.DEPT_ID;
```

2.5 WITH Clause (Common Table Expressions)

The `WITH` clause defines a subquery block that can be referenced multiple times in the main query. It improves readability and sometimes performance.

2.5.1 Example: Using WITH for Department Salary Summary

```
WITH dept_summary AS (
SELECT
DEPT_ID,
COUNT(*) AS EMP_COUNT,
AVG(SALARY) AS AVG_SALARY
FROM EMPLOYEE
GROUP BY DEPT_ID
)
SELECT
d.DEPT_NAME,
ds.EMP_COUNT,
ROUND(ds.AVG_SALARY, 2) AS AVG_SALARY
FROM DEPARTMENT d
JOIN dept_summary ds
ON d.DEPT_ID = ds.DEPT_ID
ORDER BY ds.AVG_SALARY DESC;
```

2.5.2 Example: Projects With Total Allocated Hours and Filtered Using HAVING

```
WITH proj_hours AS (
SELECT
PROJ_ID,
SUM(ALLOCATED_HOURS) AS TOTAL_HOURS
FROM EMP_PROJECT
GROUP BY PROJ_ID
```

```
)
SELECT
p.PROJ_NAME,
ph.TOTAL_HOURS
FROM PROJECT p
JOIN proj_hours ph
ON p.PROJ_ID = ph.PROJ_ID
WHERE ph.TOTAL_HOURS >= 100;
```

2.6 Connecting the Concepts

2.6.1 Putting It All Together

A realistic analytic query usually combines all three features:

- **Joins** to bring data from multiple tables.
- **GROUP BY** and aggregates to summarise.
- **Subqueries**/`WITH` to structure complex logic.

Example 2.20 (Cyber Security Projects and Average Hours Per Employee)

```
WITH proj_hours AS (
SELECT
ep.PROJ_ID,
AVG(ep.ALLOCATED_HOURS) AS AVG_HOURS_PER_EMP
FROM EMP_PROJECT ep
GROUP BY ep.PROJ_ID
)
SELECT
p.PROJ_NAME,
d.DEPT_NAME,
ph.AVG_HOURS_PER_EMP
FROM PROJECT p
JOIN DEPARTMENT d
ON p.DEPT_ID = d.DEPT_ID
JOIN proj_hours ph
ON p.PROJ_ID = ph.PROJ_ID
WHERE d.DEPT_NAME = 'Cyber Security';
```

2.6.2 Using Correlated Subqueries in UPDATE and DELETE

Correlated subqueries are often used in `UPDATE` and `DELETE` statements to modify or remove rows based on complex conditions.

Example 2.21 (Increase salary of employees above department average)

```
-- Give an extra 5% raise to employees
-- whose salary is already above their own department average
UPDATE EMPLOYEE e
SET SALARY = SALARY * 1.05
```

```
WHERE SALARY > (
SELECT AVG(e2.SALARY)
FROM EMPLOYEE e2
WHERE e2.DEPT_ID = e.DEPT_ID
);
```

Example 2.22 (Delete project assignments for low-hour allocations)

```
-- Delete rows from EMP_PROJECT where the allocated hours
-- are below the average allocation for that project
DELETE FROM EMP_PROJECT ep
WHERE ALLOCATED_HOURS < (
SELECT AVG(ep2.ALLOCATED_HOURS)
FROM EMP_PROJECT ep2
WHERE ep2.PROJ_ID = ep.PROJ_ID
);
```

2.6.3 Practice Ideas (For Brainstorming)

Students can try:

1. List departments where the **maximum** salary is above the **overall average** salary.
2. Find employees who are working on **more than one project**.
3. List projects along with **number of employees** and **total allocated hours**, showing also projects with zero employees (use outer joins).
4. Identify departments with **no projects** assigned.
5. Use `WITH` to create a re-usable summary of employee counts per department and filter on it.

——— End of the Document ———

"The purpose of a database is to help people remember things that they would otherwise forget."

– Jim Gray

.